

JJBike

1. Objetivo.

Construir um armazém de dados para que o administrador da empresa fictícia JJBike possa compreender seu negócio.

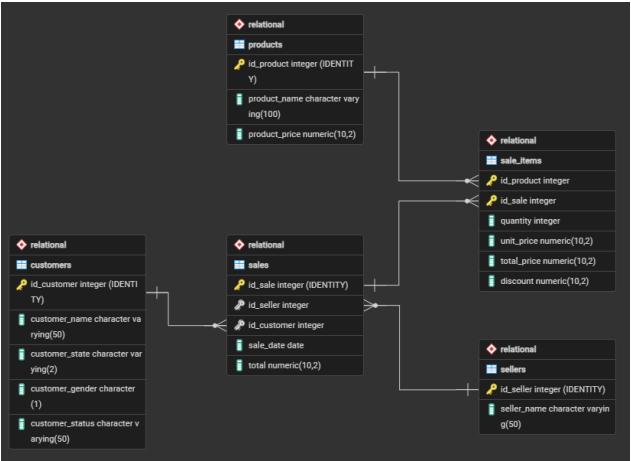
2. Modelagem.

Para criar o modelo da estrutura do banco de dados analítico (OLAP) considerando o banco de dados transacional (OLTP), o assunto que se quer analisar (o fato) e as dimensões são os diversos pontos de vista sobre o qual se quer analisar o fato (as dimensões), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

Dimensão	Atributos	Tabela de origem
dim_product (o quê?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_seller (quem?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_customer (para quem?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer
	customer_gender	customer
	customer_status	customer
	validity_start_date	-

	(versionamento)	
	validity_end_date (versionamento)	-
dim_time (quando?)	time_key (SK)	-
	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fato	Tabela consolidada	Granularidade	Atributos	Tabela de origem
fact_sales	sales	Item vendido.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



3. Ambiente transacional (OLTP) - Microsoft SQL Server Management Studio (SSMS).

3.1. Criação do schema relacional.

Foi criado o banco de dados Relational, que serve como camada de origem dos dados transacionais da empresa JJBike. Dentro dele, o arquivo 1. Scripts_Create_Table_Relational.sql criou cinco tabelas operacionais. Em todas elas, a chave primária é gerada automaticamente pela cláusula IDENTITY, eliminando a necessidade de controlar sequências externas. Todas as colunas, exceto as próprias chaves primárias, foram definidas com NOT NULL, garantindo que nenhum registro seja persistido com dados obrigatórios em aberto.

- **sellers:** Armazena os dados dos vendedores. O atributo seller_name identifica o nome do vendedor e é o único campo além da PK, refletindo o escopo intencional desta entidade no modelo transacional;
- **products:** Armazena o catálogo de produtos. O atributo product_name identifica o produto e product_price registra o preço de tabela vigente ao momento do cadastro. Ambos são obrigatórios, pois um produto sem nome ou sem preço não pode participar de um processo de venda;
- **customers:** Armazena o cadastro de clientes. O atributo customer_name identifica o cliente; customer_state registra a UF de origem (Varchar(2)); customer_gender registra o gênero em um único caractere (Char(1)); e customer_status registra o nível do plano do cliente, atributo que, por ser mutável ao longo do tempo, será o principal elemento rastreado pelo SCD Tipo 2 no ambiente analítico;
- **sales:** Representa o cabeçalho de cada venda. O atributo id_seller é uma FK que referencia Relational..sellers, e id_customer é uma FK que referencia Relational..customers, ambas obrigatórias, pois uma venda não pode existir sem um vendedor e um cliente associados. O atributo sale_date registra a data em que a transação ocorreu com tipo DATETIME, e total armazena o valor consolidado da venda;
- **sale_items:** Tabela de ligação entre sales e products, com chave primária composta por id_product e id_sale. As constraints de integridade referencial foram declaradas via ALTER TABLE após a criação das tabelas:
 - FK_SALEITEMS_PRODUCT em id_product: referencia Relational..products, impedindo que dados históricos percam a referência ao produto transacionado;
 - FK_SALEITEMS_SALE em id_sale: referencia Relational..sales, garantindo que itens sempre estejam associados a uma venda existente;
 - Os atributos quantity, unit_price, total_price e discount são as métricas de detalhe de cada item e foram declaradas NOT NULL pois alimentarão diretamente as colunas de medida da tabela fato no ambiente analítico.

3.2. População das tabelas.

Os dados foram inseridos por meio dos arquivos 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql e 6. Insert_Into_SaleItems.sql. Cada arquivo executa comandos INSERT INTO na respectiva tabela do banco Relational, informando apenas as colunas de dados, as chaves primárias são omitidas porque o mecanismo IDENTITY as atribui automaticamente e de forma sequencial a cada registro inserido.

4. Ambiente de staging (invisível/conceitual).

Não houve tratamento ou limpeza de dados brutos antes de criar as dimensões, a própria transferência direta de dados entre os schemas do PostgreSQL funcionou como o gatilho de carga.

5. Ambiente analítico (OLAP) - Microsoft SQL Server Management Studio (SSMS).

5.1. Criação do banco de dados e das tabelas dimensionais.

Foi criado o banco de dados Dimensional, que representa o Data Warehouse estruturado em modelo Star Schema. O arquivo 1. Scripts_Create_Table_Dimensional.sql criou as dimensões dim_seller, dim_customer, dim_product e dim_time, além da tabela fato fact_sales. Em todas as tabelas, a Surrogate Key (SK) é gerada automaticamente por IDENTITY, servindo como chave primária interna do DW, desacoplada dos IDs de origem do sistema transacional.

Todas as colunas foram definidas com NOT NULL, com a única exceção de validity_end_date nas dimensões com SCD Tipo 2. Esse campo permanece NULL enquanto o registro estiver ativo; só é preenchido com o momento corrente no momento em que uma mudança de atributo é detectada e uma nova versão do registro precisa ser criada. Os campos de validade são do tipo DATETIME, permitindo a diferenciação de registros processados na mesma data com precisão de milissegundos.

As tabelas dimensionais possuem a seguinte estrutura de atributos:

- **dim_seller:** seller_key é a SK gerada automaticamente e atua como PK interna do DW. id_seller preserva o ID de origem vindo de Relational.sellers, mantendo rastreabilidade entre os dois ambientes. seller_name carrega o nome do vendedor. validity_start_date recebe GETDATE() no momento em que o registro é inserido, marcando o início de sua vigência. validity_end_date permanece NULL enquanto o registro for a versão ativa;
- **dim_customer:** customer_key é a SK, PK interna do DW. id_customer preserva o ID de origem. customer_name, customer_state, customer_gender e customer_status são os atributos descritivos rastreados, qualquer alteração em qualquer um deles aciona o versionamento SCD Tipo 2. validity_start_date e validity_end_date controlam o período de vigência de cada versão do registro;
- **dim_product:** product_key é a SK. id_product preserva o ID de origem. product_name é o único atributo descritivo rastreado, uma alteração em seu valor dispara o versionamento. validity_start_date e validity_end_date funcionam da mesma forma que nas demais dimensões SCD Tipo 2;
- **dim_time:** time_key é a SK. time_date armazena a data completa e serve como campo de ligação com o sistema transacional durante a carga da fato. time_day, time_month, time_year e time_quarter são derivados de time_date e habilitam agrupamentos analíticos por granularidade temporal. time_weekday armazena o dia da semana como inteiro (1 = domingo, 7 = sábado), seguindo o padrão de DATEPART(weekday, ...) do SQL Server. Esta dimensão não possui campos de validade pois o tempo é imutável;
- **fact_sales:** sale_key é a SK da fato. seller_key, customer_key, product_key e time_key são as FKs que referenciam as respectivas dimensões, todas declaradas NOT NULL, pois cada fato precisa obrigatoriamente estar associado a todas as dimensões do modelo. quantity, unit_price, total_price e discount são as métricas de cada item vendido, também NOT NULL.

5.2. População da dimensão tempo.

A dimensão dim_time foi populada pelo arquivo 2. Insert_Into_dim_time.sql. A estratégia adotada foi a geração programática de um intervalo contínuo de datas de 1º de janeiro de 1970 a 31 de dezembro de 2030. O script é composto por três blocos encadeados:

- **Configuração do formato de data:** SET DATEFORMAT dmy define a interpretação de datas no padrão dia/mês/ano para a sessão, garantindo que as datas literais sejam lidas corretamente pelo SQL Server;
- **CTE recursiva date_sequence:** utiliza uma CTE com âncora em @start_date e recursão via DATEADD(day, 1, generated_date) até atingir @end_date. A cláusula OPTION (MAXRECURSION 0) remove o limite padrão de 100 iterações do SQL Server, permitindo que a recursão percorra todos os dias do intervalo sem interrupção;
- **INSERT final:** insere cada data gerada em dim_time, extraíndo time_day via DAY(), time_month via MONTH(), time_year via YEAR(), time_weekday via DATEPART(weekday, ...) e time_quarter via DATEPART(quarter, ...).

5.3. Extração (E) e carregamento (L) dos dados do ambiente transacional (OLTP) para o ambiente analítico (OLAP).

As cargas foram realizadas pelo arquivo 3. Script.sql, estruturado em fases sequenciais para demonstrar o funcionamento completo do SCD Tipo 2 e mecanismos avançados de integridade referencial e histórica.

5.3.1. Carga inicial das dimensões.

O padrão de carga das três dimensões com suporte a SCD Tipo 2 foi aprimorado para lidar tanto com atualizações de atributos quanto com a deleção física de registros na origem (*Hard Deletes*). Antes de cada bloco de carga, duas variáveis de controle temporal são declaradas: @dt_end captura o momento exato da execução via GETDATE() e é usada para encerrar os registros antigos ou deletados; @dt_start recebe @dt_end + 3 milissegundos via DATEADD(MILLISECOND, 3, @dt_end) e atua como validity_start_date das novas versões. O mecanismo opera em duas etapas independentes:

- **Deteção de exclusões (Deletes):** Um comando UPDATE prévio varre a dimensão e encerra a vigência (SET validity_end_date = @dt_end) de todas as linhas atualmente ativas (validity_end_date IS NULL) cujos IDs de negócio não existam mais na tabela correspondente do banco Relational;
- **Processamento do MERGE e INSERT externo:** O bloco principal executa um MERGE comparando a fonte com o destino apenas sobre as linhas ativas (T.validity_end_date IS NULL). Se os atributos divergirem (WHEN MATCHED AND), a versão atual é encerrada no destino com @dt_end. Se o ID não existir no DW (WHEN NOT MATCHED BY TARGET), o registro novo é inserido com início em @dt_start. A cláusula OUTPUT \$ACTION direciona as linhas alteradas para um INSERT externo, que filtra apenas action = 'UPDATE' para gerar a nova linha histórica ativa com validity_start_date = @dt_start e data fim nula.

5.3.2. Carga da tabela fato (mês a mês).

A tabela fact_sales é populada por um INSERT INTO ... SELECT que consolida os dados operacionais com o histórico exato do Data Warehouse. As vendas foram carregadas de forma particionada por mês (janeiro, fevereiro, março e meses subsequentes) para evidenciar a dinâmica das mutações do SCD Tipo 2.

Para evitar o problema de "fatos borradas" (*blurred facts*) — onde reprocessamentos retroativos associariam incorretamente vendas passadas a SKs novas —, o mapeamento abandonou o filtro fixo de linhas ativas e adotou junções temporais:

- **Métricas:** quantity, unit_price, total_price e discount provêm diretamente de Relational.sale_items, respeitando a granularidade do item de venda;
- **Chaves substitutas (Surrogate Keys):** As chaves seller_key, customer_key e product_key são obtidas por INNER JOIN com suas respectivas dimensões avaliando a data em que a venda ocorreu (v.sale_date). A junção valida se o fato aconteceu dentro da janela de vigência daquela versão da dimensão: v.sale_date >= validity_start_date AND (v.sale_date

`<= validity_end_date OR validity_end_date IS NULL`). Isto assegura que a fato sempre aponte para a SK que reflete o estado real da entidade no exato momento da venda;

- **time_key:** Obtida mapeando `v.sale_date` diretamente contra o atributo `time_date` da dimensão tempo (`dim_time`);
- **Filtro temporal:** A função `MONTH(v.sale_date)` restringe a massa de dados ao bloco correspondente de simulação (`= 1` para janeiro, `= 2` para fevereiro, `= 3` para março e `> 3` para o restante).

5.3.3. Simulação de mudança de status e processamento do histórico.

Para demonstrar o versionamento cronológico, duas atualizações de negócio foram simuladas na tabela origem `Relational..customers`:

- **Após a carga de janeiro:** Os clientes de ID 1 a 5 foram promovidos para o status 'Gold';
- **Após a carga de março:** O cliente de ID 3 foi promovido para o status 'Platinum'.

A cada alteração operacional, a rotina completa da dimensão (varredura de exclusão + MERGE com OUTPUT) foi reexecutada. O motor identificou a divergência em `customer_status`, invalidou a SK até então ativa gravando a data de término e gerou um novo registro com o novo status e início de vigência reajustado.

5.3.4. Verificação e auditoria do SCD Tipo 2.

Consultas analíticas de auditoria cruzando `fact_sales`, `dim_customer` e `dim_time` pelas chaves substitutas validaram a precisão do ecossistema:

- Os registros de janeiro apontam perfeitamente para as SKs antigas dos clientes 1 a 5, preservando a fotografia da época em que eles ainda não eram 'Gold';
- Os registros de fevereiro refletem as SKs intermediárias geradas após a primeira alteração (status 'Gold');
- A consulta focada em `t.time_month = 3` atesta que nenhum dos clientes mapeados comprou em março, agindo como validador de consistência;
- A auditoria final após o carregamento dos meses finais (`> 3`) exhibe a coexistência harmoniosa de fatos antigos (ligados às SKs passadas de status 'Gold' ou anteriores) e fatos novos (vinculados à SK mais recente do cliente 3 como 'Platinum'), consolidando o sucesso da rastreabilidade temporal.

5.4. KPI (indicador de performance).

Para medir a receita realizada da JJBike em relação às metas mensais, foi criada a tabela `Dimensional..kpi` pelo arquivo 4. `KPI.sql`. O script é composto por quatro estruturas principais:

- **Limpeza prévia:** o bloco `IF OBJECT_ID('Dimensional..kpi', 'U') IS NOT NULL DROP TABLE Dimensional..kpi` verifica se a tabela já existe antes de recriá-la, evitando erros em re-execuções do script;
- **Bloco SELECT:** define as três colunas que comporão a tabela: `dt.time_month` aliado como `month`, que identifica o mês de referência; `SUM(fs.total_price)` aliado como `total_revenue`, que agrega a receita efetivamente realizada somando `total_price` de todos os registros de `fact_sales` do respectivo mês; e um bloco `CASE dt.time_month` que atribui um valor de meta fixo para cada mês, de R\$ 220.000 em janeiro a R\$ 270.000 em dezembro, aliado como `target`;
- **Conversão de tipo:** o operador `CAST(... AS NUMERIC(18,2))` posicionado após o `END` do `CASE` garante que a coluna `target` seja consolidada com o mesmo tipo numérico de precisão fixa de `total_price` em `fact_sales`. Sem essa conversão, o SQL Server trataria os valores do `CASE` como inteiros, causando incompatibilidade de tipo ao calcular percentuais de atingimento;

- **Bloco FROM:** parte de Dimensional..fact_sales e realiza um INNER JOIN com Dimensional..dim_time pela condição fs.time_key = dt.time_key, relacionando cada registro da fato ao seu respectivo mês. O GROUP BY dt.time_month consolida todas as vendas de cada mês em uma única linha, e o ORDER BY dt.time_month ASC organiza o resultado em ordem cronológica crescente. O operador SELECT INTO cria fisicamente a tabela Dimensional..kpi a partir do resultado da consulta.

6. Artefatos - Power BI.

As tabelas do banco de dados Dimensional, criadas no SSMS, foram conectadas ao Power BI.

6.1. Relatórios.

6.1.1. Transformações (T) nas tabelas.

Em fact_sales, foi criada uma coluna chamada discount_percent (com duas casas decimais), com a seguinte fórmula:

- discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2);

Em dim_customer, foi criada uma coluna chamada location_map, com a seguinte fórmula:

- location_map =
SWITCH(
 'dimensional dim_customer'[customer_state],
 "AC", "Acre",
 "AL", "Alagoas",
 "AM", "Amazonas",
 "AP", "Amapa",
 "BA", "Bahia",
 "CE", "Ceara",
 "DF", "Federal District",
 "ES", "Espírito Santo",
 "GO", "Goias",
 "MA", "Maranhao",
 "MG", "Minas Gerais",
 "MS", "Mato Grosso do Sul",
 "MT", "Mato Grosso",
 "PA", "Para",
 "PB", "Paraiba",
 "PE", "Pernambuco",
 "PI", "Piaui",
 "PR", "Parana",
 "RJ", "Rio de Janeiro",
 "RN", "Rio Grande do Norte",
 "RO", "Rondonia",
 "RR", "Roraima",
 "RS", "Rio Grande do Sul",
 "SC", "Santa Catarina",

```

"SE", "Sergipe",
"SP", "Sao Paulo",
"TO", "Tocantins",
"Unknown"
) & ", Brazil".

```

6.1.2. Métricas.

Em dim_kpi, foram criadas três métricas para que, no relatório de KPI, o card tenha comportamento dinâmico de acordo com o que estiver selecionado nos botões do button slicer:

- `measure_target =`

```

IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
dim_time'[time_year]),
    SUM('dimensional kpi'[target]),
    0
).

```
- `measure_total_revenue =`

```

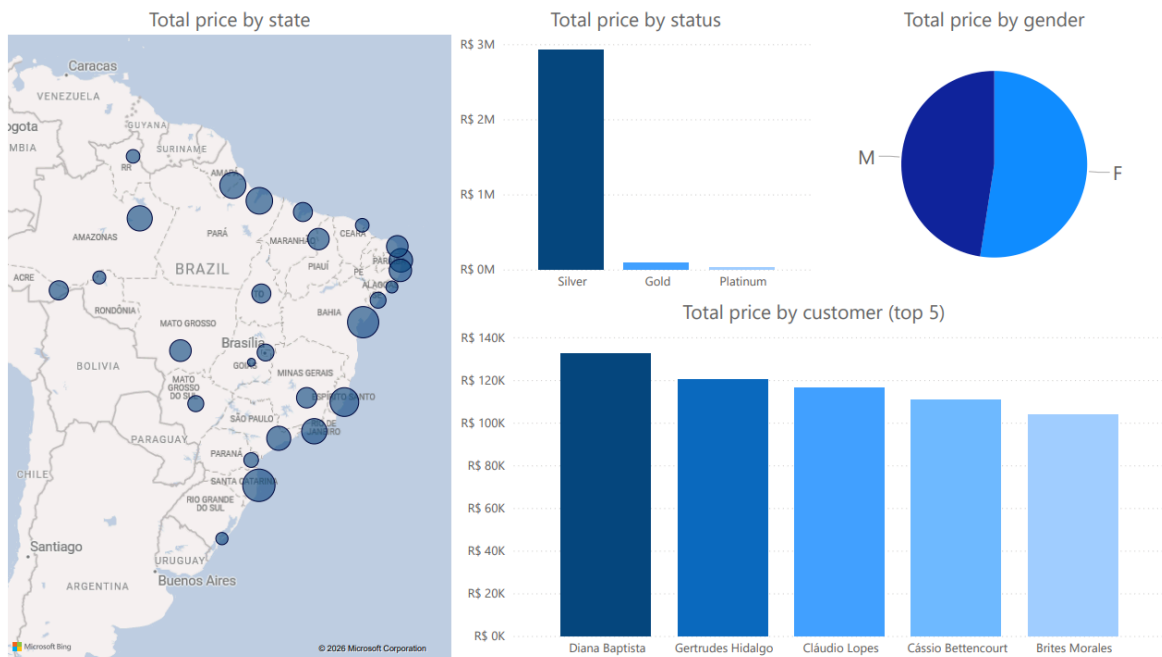
IF(
    ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional
dim_time'[time_year]),
    SUM('dimensional kpi'[total_revenue]),
    0
).

```

6.1.3. Relatórios criados.

(Clientes)

O relatório de clientes apresenta: somatório de valor total por estado, somatório de valor total por status, somatório de valor total por gênero e somatório de valor total por cliente (top 5).



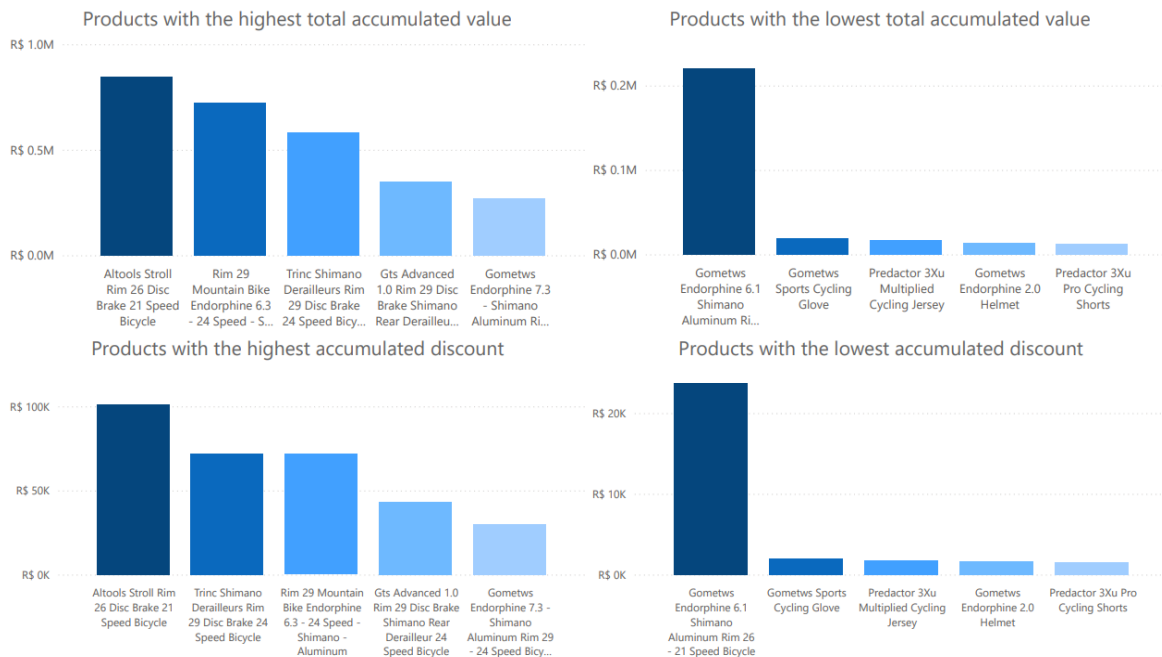
(Vendedores)

O relatório de vendedores apresenta: somatório de valor total por vendedor (top 5 melhores vendedores, top 5 piores vendedores e top 5 melhores vendedores comparados mês a mês).



(Produtos)

O relatório de produtos apresenta: somatório de valor total por produto (top 5 de produtos mais vendidos e top 5 de produtos menos vendidos) e somatório de desconto por produto (top 5 de produtos com mais desconto acumulado e top 5 de produtos com menos desconto acumulado).



6.2. Dashboards.

Foram criados os seguintes dashboards:

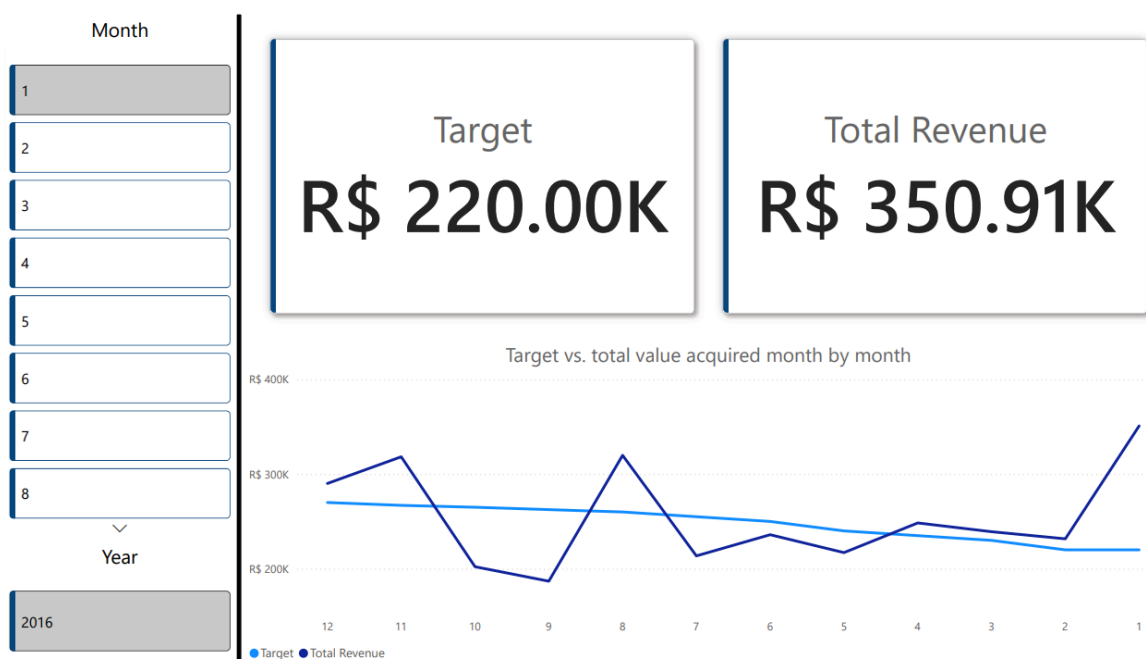
(Vendas)

O relatório de vendas apresenta: somatório de valor total de produtos vendidos (comparados mês a mês), somatório de quantidade de produtos vendidos (comparados mês a mês), somatório de desconto de produtos vendidos (comparados mês a mês) e cards de produtos, vendedores e clientes para filtragem.



(KPI)

O relatório de KPI apresenta: meta, valor total adquirido, somatório de meta e valor total adquirido (comparados mês a mês) e cards de mês e ano para filtragem.



Arquivo gerado disponível em '2. Artifacts/1. Reports+Dashboards/'.

7. Artefato - Microsoft Visual Studio.

7.1. Cubo.

As tabelas do Dimensional banco de dados foram estruturadas e hospedadas em uma instância completa do SQL Server (Developer Edition), garantindo suporte nativo às funcionalidades de Business Intelligence (BI);

Uma Data Source View (DSV) foi criada, um ambiente visual onde as tabelas fato e de dimensão foram mapeadas (excluindo a tabela kpi), gerando um modelo lógico no formato Star Schema;

As seguintes dimensões foram criadas:

- **dim_customer:** customer_key, customer_name, customer_gender, customer_state, e customer_status;
- **dim_product:** product_key e product_name;
- **dim_time:** time_key, time_year, time_quarter, time_month, e time_day;
- **dim_seller:** seller_key e seller_name.

As seguintes hierarquias foram criadas a partir das dimensões:

- **Hierarquia Geográfica:** customer_state → customer_name (dim_customer).
 - **Obs.:** Na aba Attribute Relationships, a ordem deve ser: Customer Key → Customer Name → Customer State.
- **Hierarquia Temporal:** time_year → time_quarter → time_month → time_day (dim_time).

- **Obs.:** Na aba Attribute Relationships, a ordem deve ser: Time Key → Time Day → Time Month → Time Quarter → Time Year.
- **Observações:**
 - customer_gender e customer_status (de dim_customer) devem ficar livres no painel lateral como Atributos de Dimensão Soltos, pois inseri-los dentro de Geographic Hierarchy implicaria que são parte de uma subdivisão geográfica;
 - dim_seller e dim_product possuem apenas um atributo além de suas chaves, portanto não geraram hierarquias, e seus atributos tornaram-se Atributos de Dimensão Soltos;
 - Para evitar erros de chave duplicada no mecanismo do Analysis Services durante o processamento de dim_time, as propriedades KeyColumns foram personalizadas como chaves compostas, vinculando time_quarter com time_year, time_month com time_year, e time_day com time_month e time_year, garantindo a unicidade e a integridade da árvore de relacionamentos dos atributos.

O cubo foi estruturado a partir da fact_sales tabela fato. Durante o processo, as chaves de relacionamento e os campos analiticamente incorretos (como unit_price, cuja agregação por soma não faz sentido de negócio) foram ignorados. Apenas as métricas reais de desempenho foram mantidas (quantity, total_price, discount, e a contagem automática de registros);

Um deploy bem-sucedido do projeto foi realizado na instância local do servidor Analysis Services, compilando as estruturas e carregando os dados na memória;

Representação visual do cubo criado:

Customer State	Customer Name	Time Year	Time Quarter	Time Month	Time Day	Quantity	Discount	Total Price
AC	Alarico Quinterno	2016	4	12	10	1	60.45	155
AC	Antônio Sobral	2016	3	9	17	1	1593.36	8852
AC	Antônio Sobral	2016	4	10	16	2	1741.25	7935.6
AC	Basilio Soares	2016	1	3	29	1	4.65	155
AC	Basilio Soares	2016	2	4	17	9	879.08	22167.65
AC	Basilio Soares	2016	2	4	18	3	125.1	3289
AC	Basilio Soares	2016	2	5	14	1	64.07	2135.52
AC	Basilio Soares	2016	2	6	6	1	650.93	6509.3
AC	Basilio Soares	2016	2	6	24	1	16.92	188
AC	Basilio Soares	2016	3	7	5	2	775.25	7793
AC	Carla Briones	2016	4	12	5	1	587.75	2260.58
AC	Ester Castanho	2016	3	9	26	5	2327.33	15416.98
AC	Evaristo Bahia	2016	4	10	10	3	692.17	4836.7
AC	Irene Villanueva	2016	4	11	12	3	1654.34	7183.75
AL	Cid Pardo	2016	3	8	21	3	1769.02	13295
AL	Cid Pardo	2016	4	10	18	3	2430.31	13522.61
AM	Deise Farias	2016	3	8	22	3	1231.04	8976.8
AM	Deise Farias	2016	3	9	7	1	1472.16	9201
AM	Diana Baptista	2016	1	2	12	4	79.67	11112.58
AM	Diana Baptista	2016	1	2	13	1	1.69	169.2
AM	Diana Baptista	2016	1	3	15	4	285.75	7205.75
AM	Diana Baptista	2016	2	4	2	1	56.31	5631.01

Arquivos gerados disponíveis em '2. Artifacts/2. Cube/'.